Programowanie obiektowe i inżynieria oprogramowania

Lab 4.1:

- deklaracja a definicja zmiennej (*extern*) i funkcji,
- czas życia zmiennych, przysłanianie zmiennych,
- argumenty funkcji, referencja, wskaźniki,
- konwersja typu zmiennej.

Akademia Górniczo-Hutnicza im. Stanisława Staszica w Krakowie
AGH University of Science and Technology

Prowadzący: dr inż. Jakub Bryła
2024

Deklaracja a definicja zmiennej (*extern*) i funkcji

Deklaracja a definicja zmiennej (*extern*) i funkcji

Linia 5: deklaracja funkcji
Linia 9: wywołanie funkcji
Linia 14-17: definicja funkcji

main.cpp | conversion.h

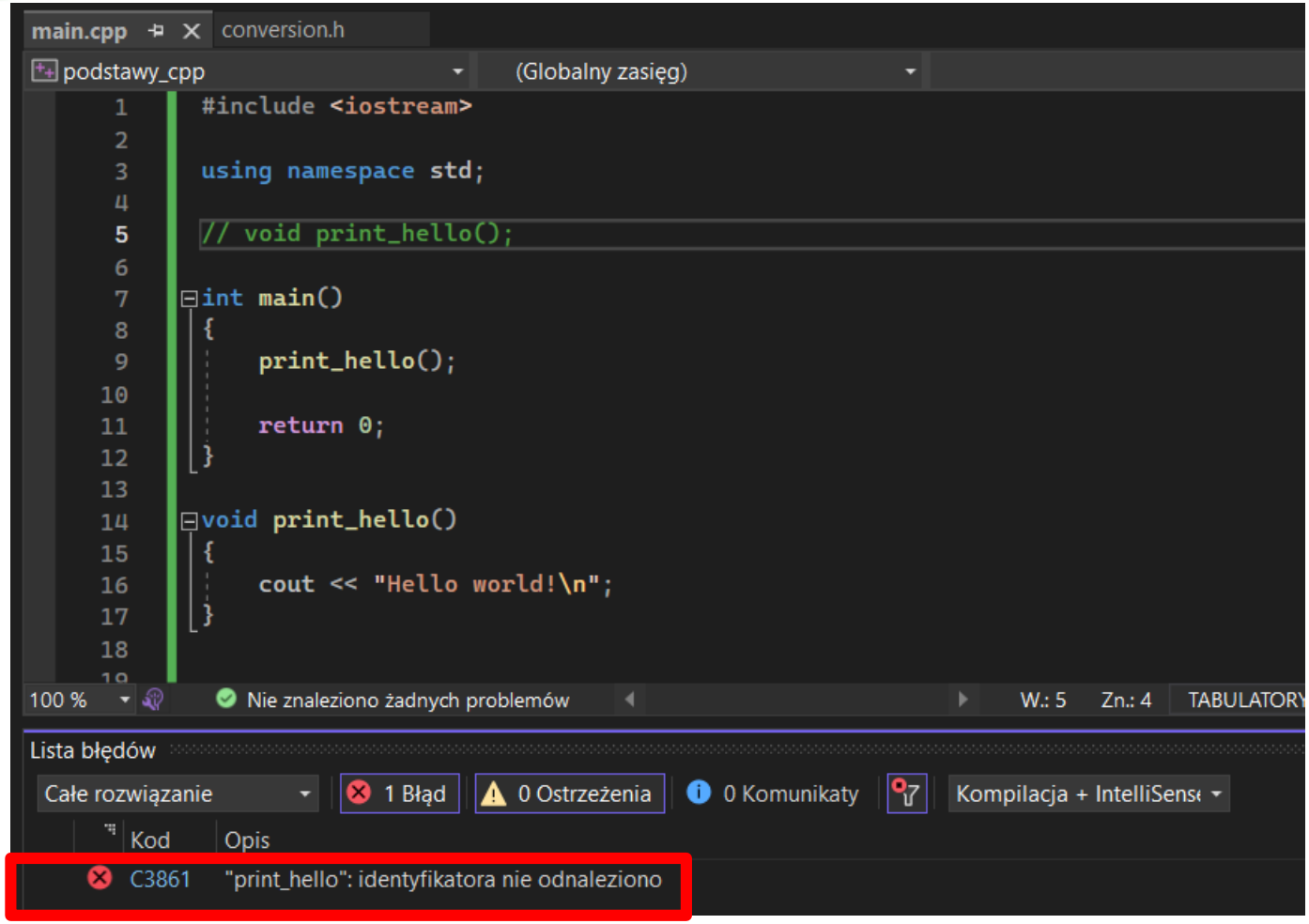
podstawy_cpp | (Globalny zasięg)

```
1  #include <iostream>
2
3  using namespace std;
4
5  void print_hello();
6
7  int main()
8  {
9      print_hello();
10
11     return 0;
12 }
13
14 void print_hello()
15 {
16     cout << "Hello world!\n";
17 }
18
```

Konsola debugowania programu Microsoft Visual Studio

```
Hello world!

C:\Users\jakbr\source\repos\podstawy_cpp\x64\Deb
Aby automatycznie zamknąć konsolę po zatrzymaniu
ie -> Automatycznie zamknij konsolę po zatrzymar
Naciśnij dowolny klawisz, aby zamknąć to okno...
```

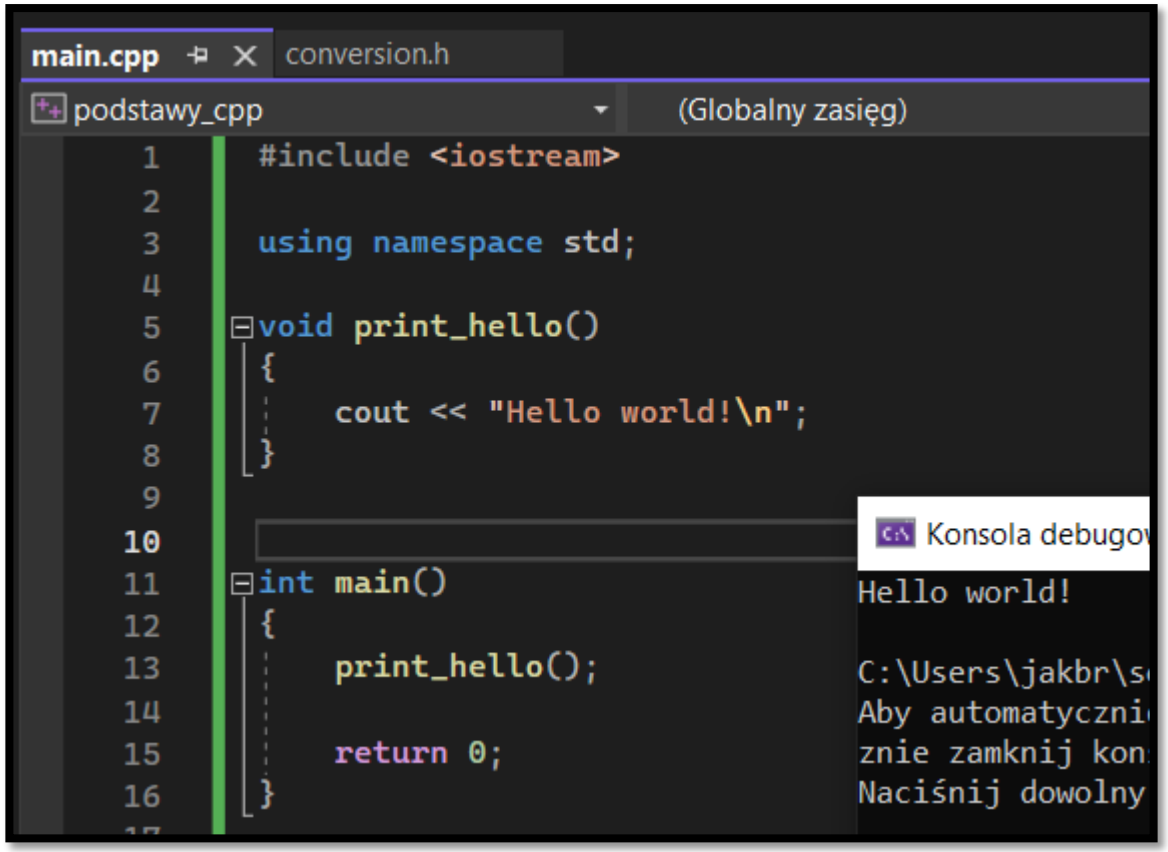
Deklaracja a definicja zmiennej (*extern*) i funkcji



Przed wywołaniem funkcji musi znajdować się **MINIMUM** jej deklaracja!

Plik kompilowany jest linia po linii, od góry do dołu. Przed wywołaniem funkcji kompilator musi wiedzieć co to za funkcja (jakie argumenty przyjmuje i co zwraca).

Deklaracja a definicja zmiennej (*extern*) i funkcji



Definicja to deklaracja + kod funkcji, tak więc kod w tym przypadku tym bardziej zadziała. Kompilator dostał wymaganą deklarację funkcji (i coś więcej) przed jej pierwszym wywołaniem.

Deklaracja a definicja zmiennej (*extern*) i funkcji

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg)

```
1  #include <iostream>
2
3  using namespace std;
4
5  int i_global;
6
7  int main()
8  {
9      int i_func = 5;
10
11     cout << "int i_global=" << i_global
12         << "\nint i_func=" << i_func << "\n\n";
13
14     return 0;
15 }
```

Konsola debugowania programu Microsoft Visual Studio

```
int i_global=0
int i_func=5
```

W linijkach 5 i 9 znajdują się **definicje** zmiennych.

W linii 5 definiowana jest zmienna globalna, która inicjalizowana jest automatycznie wartością 0.

W linii 9 definiowana jest zmienna lokalna (wew. funkcji main). W starszych środowiskach programowania istniała możliwość zdefiniowana zmiennej lokalnej bez podania wartości – w takim wypadku do zmiennej przypisywany był adres pamięci, bez jej czyszczenia/nadpisywania (zmienna przyjmowała napotkaną wartość w przyznanej komórce pamięci). Zgodnie ze standardem C++ dla zmiennych lokalnych nie następuje automatyczne przypisanie wartości 0!

Deklaracja a definicja zmiennej (*extern*) i funkcji

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```
1  #include <iostream>
2
3  using namespace std;
4
5  extern int i_ext;
6  int i_global;
7
8  int main()
9  {
10     int i_func = 5;
11
12     cout << "int i_global=" << i_global << endl;
13     cout << "int i_func=" << i_func << endl;
14     cout << "extern int i_ext=" << i_ext << endl;
15
16     cout << endl;
17     return 0;
18 }
```

Lista błędów

Całe rozwiązanie | 2 Błędy | 0 Ostrzeżenia | 0 Komunikaty

Kod	Opis
LNK2001	nierozpoznany symbol zewnętrzny "int i_ext" (?i_ext@@3HA)
LNK1120	liczba nierozpoznanych elementów zewnętrznych: 1

Kompilator zwrócił błąd, ponieważ zmienna *i_ext* nie została utworzona. W linii 5 poddano jedynie deklaracje (*extern*), bez zarezerwowania pamięci na zmienną.

Deklaracja a definicja zmiennej (*extern*) i funkcji

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```
1   #include <iostream>
2
3   using namespace std;
4
5   extern int i_ext;
6   int i_global;
7
8   int main()
9   {
10      int i_func = 5;
11      int i_ext = 10;
12
13      cout << "int i_global=" << i_global << endl;
14      cout << "int i_func=" << i_func << endl;
15      cout << "extern int i_ext=" << i_ext << endl;
16
17      cout << endl;
18      return 0;
19  }
```

Konsola debugowania programu M

```
int i_global=0
int i_func=5
extern int i_ext=10
```

Deklaracja zmiennej występuje tylko w jednym przypadku – użycia słówka *extern*.

Deklaracja – określenie nazwy i typu zmiennej: linia 5.

Definicja – utworzenie (zarezerwowanie miejsca w pamięci) zmiennej o podanej nazwie i typie: linia 11.

Czas życia zmiennych, przysłanianie zmiennych

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;


int main()
{
    int tabela[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    int i = 0;

    while (tabela[i] < 5)
    {
        i += 1;
    }

    for (int i=0; i < 9; i++)
    {
        cout << tabela[i] << ", ";
    }

    cout << endl;

    for (i; i < 9; i++)
    {
        cout << tabela[i] << ", ";
    }

    cout << endl;
    return 0;
}
```

Konsola debugowania programu M

[redacted]

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;


int main()
{
    int tabela[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9 };
    int i = 0;

    while (tabela[i] < 5)
    {
        i += 1;
    }

    for (int i=0; i < 9; i++)
    {
        cout << tabela[i] << ", ";
    }

    cout << endl;

    for (i; i < 9; i++)
    {
        cout << tabela[i] << ", ";
    }

    cout << endl;
    return 0;
}
```

Konsola debugowania programu M

```
1, 2, 3, 4, 5, 6, 7, 8, 9,
5, 6, 7, 8, 9,
```

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;


int main()
{
    int i = 10;
    cout << "int i=" << i << endl;

    {
        cout << "\nint i=" << i << endl;
        i = 7;
        cout << "int i=" << i << endl;

        int i = 5;
        cout << "int i=" << i << endl;

        i = 3;
        cout << "int i=" << i << endl;
    }

    cout << "\nint i=" << i << endl;

    cout << endl;
    return 0;
}
```

Konsola debugowania

[redacted]

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;


int main()
{
    int i = 10;
    cout << "int i=" << i << endl;

    {
        cout << "\nint i=" << i << endl;
        i = 7;
        cout << "int i=" << i << endl;

        int i = 5;
        cout << "int i=" << i << endl;

        i = 3;
        cout << "int i=" << i << endl;
    }

    cout << "\nint i=" << i << endl;

    cout << endl;
    return 0;
}
```

Konsola debugowania

```
int i=10

int i=10
int i=7
int i=5
int i=3

int i=7
```

Czas życia zmiennej zazwyczaj ograniczony jest do bloku **{ }**, w którym zostają zdefiniowana.

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | main()

```cpp
#include <iostream>

using namespace std;

void add(int num1, int num2)
{
    int num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;
    cout << num1 << " + " << num2 << " = ???\n";

    add(num1, num2);

    cout << num1 << " + " << num2 << " = ";
    // BŁĄD: zmienna num nie istnieje w funkcji main
    // cout << num << "\n";

    cout << endl;
    return 0;
}
```

Poświęcimy teraz nieco uwagi funkcji do dodawania dwóch zmiennych do siebie.

Konsola debugowania

[redacted]

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | main()

```cpp
#include <iostream>

using namespace std;

void add(int num1, int num2)
{
    int num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;
    cout << num1 << " + " << num2 << " = ???\n";

    add(num1, num2);

    cout << num1 << " + " << num2 << " = ";
    // BŁĄD: zmienna num nie istnieje w funkcji main
    // cout << num << "\n";

    cout << endl;
    return 0;
}
```

Konsola debugowania

```
5 + 8 = ???
5 + 8 = 13

5 + 8 =
```

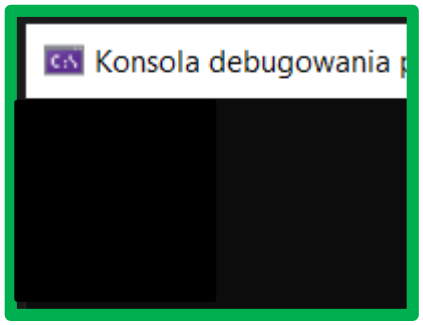
Funkcja nie zwraca wyniku – wyświetla go tylko na wyjście, w tym przypadku do konsoli.

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | main()

```cpp
#include <iostream>

using namespace std;

int num;

void add(int num1, int num2)
{
    int num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

Najprostszym „uniwersalnym” wyjściem jest stworzenie zmiennej globalnej, w której zapiszemy wynik działania funkcji.



main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | main()

```cpp
#include <iostream>

using namespace std;

int num;

void add(int num1, int num2)
{
    int num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

W tym przypadku mamy jednak jeszcze problem z przysłonięciem nazwy zmiennej globalnej.

Konsola debugowania

```
5 + 8 = 0

5 + 8 = 13

5 + 8 = 0
```

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;

int num;

void add(int num1, int num2)
{
    cout << "int num=" << num << endl;
    int num = num1 + num2;
    cout << "int num=" << num << endl;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;
    [redacted]

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

Mały dowód, że zmienna globalna jest dostępna w funkcji.

Konsola debugowania programu

[redacted]

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;

int num;

void add(int num1, int num2)
{
    cout << "int num=" << num << endl;
    int num = num1 + num2;
    cout << "int num=" << num << endl;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;
[redacted]

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

Konsola debugowania programu

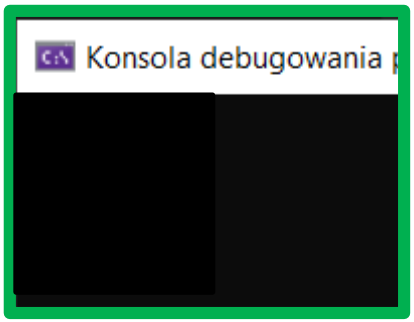
```
5 + 8 = 0

int num=0
int num=13
5 + 8 = 13

5 + 8 = 0
```

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | add(int nu

```cpp
#include <iostream>

using namespace std;

int num;

void add(int num1, int num2)
{
    // int num = num1 + num2;
    num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

Skorzystanie ze zmiennej globalnej wewnątrz funkcji.



main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | add(int nu

```cpp
#include <iostream>

using namespace std;

int num;

void add(int num1, int num2)
{
    // int num = num1 + num2;
    num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";
}

int main()
{
    int num1 = 5;
    int num2 = 8;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

Konsola debugowania

```
5 + 8 = 0

5 + 8 = 13

5 + 8 = 13
```

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | add(int num

```cpp
#include <iostream>

using namespace std;

int add(int num1, int num2)
{
    int num = num1 + num2;
    // num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    return num;
}

int main()
{
    int num1 = 5;
    int num2 = 8;
    int num = 0;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    num = add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

Korzystanie ze zmiennych globalnych to zawsze ostateczność.

Lepszym pomysłem jest zwrócenie wartości przez funkcje.

Konsola debugowania

[redacted]

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg) | add(int num

```cpp
#include <iostream>

using namespace std;

int add(int num1, int num2)
{
    int num = num1 + num2;
    // num = num1 + num2;
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    return num;
}

int main()
{
    int num1 = 5;
    int num2 = 8;
    int num = 0;

    cout << num1 << " + " << num2 << " = " << num << "\n\n";
    num = add(num1, num2);
    cout << num1 << " + " << num2 << " = " << num << "\n\n";

    cout << endl;
    return 0;
}
```

Konsola debugowania

```
5 + 8 = 0

5 + 8 = 13

5 + 8 = 13
```

```cpp
#include <iostream>

using namespace std;

void sort(int num1, int num2)
{
    if (num1 < num2)
    {
        cout << "num2 > num1" << endl;

        int temp = num1;
        num1 = num2;
        num2 = temp;
    }

    // return num1, num2;  // BŁĄD
}

int main()
{
    int num1 = 3;
    int num2 = 7;

    sort(num1, num2);
    cout << "W kolejnosci malejacej: " << num1 << ", " << num2 << endl;
}
```

Co jednak gdybyśmy chcieli zwrócić z funkcji liczby w kolejności malejącej?

```
Konsola debugowania programu Microsoft Visual Studio
num2 > num1
W kolejnosci malejacej: 3, 7
```

C++ nie pozwala na zwrócenie z funkcji więcej niż jednej wartości.

Argumenty funkcji, referencja, wskaźniki

Referencja

main.cpp conversion.h

podstawy_cpp (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;

void sort(int &num1, int &num2)
{
    if (num1 < num2)
    {
        cout << "num2 > num1" << endl;

        int temp = num1;
        num1 = num2;
        num2 = temp;
    }

    // return num1, num2;  // BŁĄD
}

int main()
{
    int num1 = 3;
    int num2 = 7;

    sort(num1, num2);
    cout << "W kolejnosci malejacej: " << num1 << ", " << num2 << endl;
}
```

Rozwiązaniem jest przekazanie parametrów funkcji przez ***referencje***. W takim przypadku działamy na oryginalnych zmiennych.

Konsola debugowania programu Microsoft Visual Studio

```
num2 > num1
W kolejnosci malejacej: 7, 3
```

Czemu to rozwiązanie nie jest najlepsze?

W przypadku rozbudowanego programu, gdzie implementacje funkcji pochowane są w osobnych plikach, programista nie widzi w miejscu wywołania funkcji, że zmiany zostaną wykonane na oryginalnych zmiennych.

Wskaźniki

```
main.cpp   conversion.h
podstawy_cpp                    (Globalny zasięg)
1   #include <iostream>
2
3   using namespace std;
4
5
6
7   int main()
8   {
9       int num = 3;
10
11      cout << "Wartosc: " << num << endl;
12      cout << "Adres pamieci: " << &num << endl;
13      cout << endl;
14
15      int* wsk = &num;
16      cout << "Adres pamieci: " << wsk << endl;
17      cout << "Wartosc: " << *wsk << endl;
18      cout << endl;
19  }
```

Wskaźniki służą do przechowywania adresu zmiennej określonego typu.

```
Konsola debugowania programu Microsoft Visual Studio
Wartosc: 3
Adres pamieci: 000000D8F70FF994

Adres pamieci: 000000D8F70FF994
Wartosc: 3
```

- &num → adres zmiennej
- int *wsk → tworzenie wskaźnika o nazwie *wsk* do zmiennej typu *int*
- wsk → przechowywany adres we wskaźniku
- *wsk → wartość przechowywana pod adresem, który przechowuje wskaźnik

Wskaźniki

main.cpp conversion.h

podstawy_cpp (Globalny zasięg) m

```cpp
#include <iostream>

using namespace std;

void sort(int* num1, int* num2)
{
    if (*num1 < *num2)
    {
        cout << "num2 > num1" << endl;

        int temp = *num1;
        *num1 = *num2;
        *num2 = temp;
    }

    // return num1, num2;  // BŁĄD
}

int main()
{
    int num1 = 3;
    int num2 = 7;

    sort(&num1, &num2);
    cout << "W kolejnosci malejacej: " << num1 << ", " << num2 << endl;
}
```

Tym samym za pomocą wskaźnika możemy przekazać adres zmiennej do funkcji.

Konsola debugowania programu Microsoft Visual Studio

```
num2 > num1
W kolejnosci malejacej: 7, 3
```

Wskaźniki

main.cpp conversion.h

podstawy_cpp (Globalny zasięg) main()

```cpp
#include <iostream>

using namespace std;

void sort(int* num1, int* num2)
{
    if (*num1 < *num2)
    {
        cout << "num2 > num1" << endl;
        cout << "Adres zmiennej: " << num1 << ", a wartosc: " << *num1 << endl;
        cout << "Adres zmiennej: " << num2 << ", a wartosc: " << *num2 << endl;
        cout << endl;

        int temp = *num1;
        *num1 = *num2;
        *num2 = temp;
    }

    // return num1, num2;  // BŁĄD
}

int main()
{
    int num1 = 3;
    int num2 = 7;
    cout << "Adres zmiennej: " << &num1 << ", a wartosc: " << num1 << endl;
    cout << "Adres zmiennej: " << &num2 << ", a wartosc: " << num2 << endl;
    cout << endl;

    sort(&num1, &num2);
    cout << "W kolejnosci malejacej: " << num1 << ", " << num2 << endl;
}
```

Dowód, że funkcja działa na oryginalnych zmiennych.

Konsola debugowania programu Microsoft Visual Studio

```
Adres zmiennej: 0000003D220FF604, a wartosc: 3
Adres zmiennej: 0000003D220FF624, a wartosc: 7

num2 > num1
Adres zmiennej: 0000003D220FF604, a wartosc: 3
Adres zmiennej: 0000003D220FF624, a wartosc: 7

W kolejnosci malejacej: 7, 3
```

Wskaźniki

main.cpp | conversion.h

podstawy_cpp | (Globalny zasięg)

```cpp
#include <iostream>

using namespace std;

void sort(int num1, int num2)
{
    if (num1 < num2)
    {
        cout << "num2 > num1" << endl;
        cout << "Adres zmiennej: " << &num1 << ", a wartosc: " << num1 << endl;
        cout << "Adres zmiennej: " << &num2 << ", a wartosc: " << num2 << endl;
        cout << endl;

        int temp = num1;
        num1 = num2;
        num2 = temp;
    }

    // return num1, num2;  // BŁĄD
}

int main()
{
    int num1 = 3;
    int num2 = 7;
    cout << "Adres zmiennej: " << &num1 << ", a wartosc: " << num1 << endl;
    cout << "Adres zmiennej: " << &num2 << ", a wartosc: " << num2 << endl;
    cout << endl;

    sort(num1, num2);
    cout << "W kolejnosci malejacej: " << num1 << ", " << num2 << endl;
}
```

Dowód, że funkcja działa na kopiach zmiennych, gdy nie posługujemy się referencją lub wskaźnikami.

Konsola debugowania programu Microsoft Visual Studio

```
Adres zmiennej: 000000D60F98FA84, a wartosc: 3
Adres zmiennej: 000000D60F98FAA4, a wartosc: 7

num2 > num1
Adres zmiennej: 000000D60F98FA60, a wartosc: 3
Adres zmiennej: 000000D60F98FA68, a wartosc: 7

W kolejnosci malejacej: 3, 7
```